2.2.3 Metropolis-Hasting algorithm

- Metropolis–Hastings is a specific MCMC algorithm used to build the Markov chain.
- It works as follows:

1. Start at an initial value.
2. Propose a new value using a simple distribution (e.g., Gaussian noise).
3. Compute an acceptance probability based on how likely the new value is compared to the current one.
4. Accept or reject the proposal:
    - If accepted -> move to the new state
    - If rejected -> stay at the current state
5. Repeat many times to build a chain.

At the end of the procedure, we obtain a sequence of values, often called a sample chain or trace. If the algorithm has been applied correctly, this sequence provides an approximation to the posterior distribution

Refer Chapter2 Python Lab - Exercise 3

Imagine that you are walking in a foggy mountain landscape. Your goal is to spend more time in higher regions, where the elevation represents higher probability. However, because of the fog, you cannot see the entire landscape. You can only evaluate the height of your current position and nearby locations.

At each step, you choose a new position randomly. This is called a proposal. Once you move to this proposed location, you compare it with your current position. If the new location is higher, you usually accept it and move there. If the new location is lower, you may still accept it, but only with a certain probability. This probabilistic acceptance allows you to occasionally move downhill so that you do not get stuck in local peaks.

Over time, this process ensures that you spend more time in high regions of the landscape. As a result, the sequence of visited points represents samples from the target probability distribution.

Limitations of Metropolis-Hasting algorithm

- Despite its simplicity, Metropolis–Hastings has important limitations.
- The movement is essentially a random walk, which makes it inefficient.
- If the steps you take are very small, then the exploration becomes extremely slow because you only move short distances each time.

- If the steps are too large, then most proposals are rejected, which also slows down the process.
- In high-dimensional problems, this becomes even worse, as the algorithm behaves like someone wandering blindly without direction.
- This leads to slow convergence and highly correlated samples. That is if you take small steps, where you are now is very similar to where you were one step ago. And also similar to where you were two steps ago. If samples are highly correlated, then: Knowing θ_t gives you a strong idea of θ_{t+1}. Then there is little new information in each step
- In extreme cases, the chain "creeps" slowly through the space and therefore many samples look almost identical.

2.2.4 Hamiltonian Monte Carlo (HMC)

- Hamiltonian Monte Carlo is a Markov Chain Monte Carlo method that improves sampling efficiency by using gradient information to generate proposals through smooth trajectories instead of random moves.
- It works as follows:

1. Start at an initial value of the parameter θ.
2. Generate a random momentum value p sampling from a Gaussian distribution.

$$p \sim N(0, M)$$

where M is the mass matrix that controls the spread of the momentum. This acts as a random "push" that gives the system a direction and speed.

3. Simulate a smooth trajectory of the system using the gradient of the negative log-posterior. This is done using multiple small updates with a step size and number of steps (leapfrog integration)
4. This produces a proposed new state (θ', p').
5. Compute an acceptance probability based on the difference in total energy (Hamiltonian) between the current state and the proposed state.
6. Accept or reject the proposal:

- If accepted -> move to the new state θ'
- If rejected -> stay at the current state θ

7. Repeat the process many times to generate a Markov chain of samples.

- Instead of moving randomly, HMC allows the sampler to move in a more structured and informed way.
- To understand this, imagine replacing the person walking in the fog with a frictionless ball placed on the landscape.
- If you give this ball a push, it will roll smoothly across the surface, following the shape of the terrain.

Now:

- The ball does not jump randomly
- It rolls smoothly along the surface
- It follows the curvature of the terrain
- In this analogy, the landscape represents the negative log of the probability distribution (potential energy), where lower height corresponds to higher probability density.
- Instead of making random jumps, the movement follows a continuous path. The ball travels along smooth trajectories that are influenced by the slope of the landscape.
- This slope is given by the **gradient**, which is a generalization of the derivative to multiple dimensions. The gradient indicates the direction of the steepest increase in the distribution.
- Conceptually, this is achieved by introducing an additional variable called momentum. The state of the system now includes both position and momentum.
- Position represents the parameters θ)
- Momentum represents a velocity term (p)
- The movement is governed by physical laws, which ensure that the trajectory follows the contours of the distribution.

Advantages of HMC

- The use of gradient information allows the sampler to move efficiently across the distribution.
- The movement is smooth and directed, which reduces random zig-zagging behaviour.
- The sampler can travel long distances in a single iteration.
- As a result, the acceptance probability is typically higher compared to Metropolis–Hastings.
- Although each step requires more computation due to gradient calculations, the overall efficiency is often better, especially for complex or high-dimensional problems.

Limitations of HMC

- HMC requires the computation of gradients, which increases the computational cost per step.
- It also requires the user to specify key parameters:
- Step size
- Number of steps (trajectory length)
- The step size controls how accurately the continuous motion is simulated. Since the motion is approximated numerically, this parameter is necessary.
- If the step size is too small, the algorithm becomes slow.
- If the step size is too large, the simulation becomes inaccurate and proposals are often rejected.
- The number of steps determines how long the trajectory is followed:
- Too few steps result in poor exploration.
- Too many steps lead to unnecessary computation and possible inefficiency.
- Selecting appropriate values for these parameters is not straightforward.
- It often requires trial and error and some level of user experience.
- For this reason, HMC is less straightforward to implement and tune in practice compared to simpler methods such as Metropolis–Hastings, as it requires careful selection of hyperparameters and gradient evaluations, making it less readily applicable as a general-purpose sampling method without additional tuning or expertise.
- finally, we estimate the posterior using the sample frequency / empirical distribution of the generated sequence.
- HMC is powerful and efficient, especially in difficult problems.
- But it is less simple and less automatic than basic methods like Metropolis–Hastings.

2.2.5 No-U-Turn Sampler (NUTS)

- The No-U-Turn Sampler (NUTS) is an extension of Hamiltonian Monte Carlo that automatically decides how long to simulate the trajectory, removing the need to manually choose the number of steps.
- It works as follows:

1. Start at an initial value of the parameter θ.
2. Generate a random momentum p from a Gaussian distribution:

$$p \sim N(0, M)$$

3. Start building a trajectory using gradient information (similar to HMC with small steps).
4. Extend the trajectory forward and backward step by step.
5. After each expansion, check the U-turn condition:
6. Stop if the path starts turning back toward the starting point.
7. From all points in the trajectory, randomly select one as the proposed new state.
8. Compute an acceptance probability using the Hamiltonian (energy difference).
9. Accept or reject the proposal:
    - If accepted -> move to the new state θ'
    - If rejected -> stay at the current state θ
10. Repeat the process to generate a Markov chain of samples.

Strengths of NUTS

- Automatically chooses the trajectory length (no need to set number of steps).
- Reduces the need for manual tuning.
- Works well in complex and high-dimensional models

Limitations of NUTS

- More computationally complex than HMC.
- Still requires tuning of step size.
- Harder to understand and implement compared to simpler methods